Wcb

Programmer's guide

2026-10-27

Contents

index **3**
wcb **3**
wcbRef **15**
License **25**

index

Keywords: index

The Widget Callback Package Wcb 4.2

by

Csaba Nemethi
csaba.nemethi@t-online.de

Contents

Wcb Programmer's Guide

Wcb Command Reference

wcb

Keywords: wcb

Wcb Programmer's Guide

For Wcb Version 4.2

by

Csaba Nemethi
csaba.nemethi@t-online.de

Contents

Overview
- Some Common Problems
- The Wcb Package
- How to Get It?
- How to Install It?
- How to Use It?

Examples
- Some before-insert Callbacks for entry Widgets
- A selset Callback for a listbox Widget
- An activate Callback for a listbox Widget
- Seven Callbacks for a text Widget

Overview

Some Common Problems

Many Tcl/Tk programmers are confronted with questions like the following:

- How to restrict the set of characters that the user can type or paste into a Tk or Ttk entry, BWidget Entry, Tk or Ttk spinbox, Ttk combobox, text, or ctext widget?
- How to manipulate the user input characters before they are inserted into one of these widgets? In the case of a text or ctext widget: How to change the font, colors, or other attributes of the input characters?
- How to set a limit for the number of characters that can be typed or pasted into a Tk or Ttk entry, BWidget Entry, Tk or Ttk spinbox, or Ttk combobox widget?
- How to protect some parts of the text contained in a Tk or Ttk entry, BWidget Entry, Tk or Ttk spinbox, Ttk combobox, text, or ctext widget from being changed by the user?
- How to define notifications to be triggered automatically after text is inserted into or deleted from one of these widgets?
- How to define some actions to be invoked automatically whenever the insertion cursor in a Tk or Ttk entry, BWidget Entry, Tk or Ttk spinbox, Ttk combobox, text, or ctext widget is moved?
- How to define a command to be called automatically when selecting a listbox element, a tablelist row or cell, a Ttk treeview item, or a range of characters in a text or ctext widget?
- How to protect any or all items/elements of a listbox, tablelist, or Ttk treeview, or a range of characters in a text or ctext widget from being selected?

In most books, FAQs, newsgroup articles, and widget sets, you can find *individual* solutions to some of the above problems by means of widget bindings. This approach is based on adding new binding tags or modifying some of the existing ones, which quite often proves to be incomplete.

The Tk core addresses just a few of the above problems: In Tk 8.1 the `<<ListboxSelect>>` virtual event for listbox widgets was introduced, Tk versions 8.3 and higher support widget options for entry validation, and the spinbox widget (introduced in Tk 8.4) provides the same validation facility. Finally, Tk versions 8.4 and higher support the `disabled` state for listbox widgets, as well as the `modified` flag, the `<<Modified>>` and `<<Selection>>` virtual events, and an undo/redo mechanism for text widgets. However, also these improvements are of *individual* nature.

The Wcb Package

The **W**idget **c**all**b**ack package Wcb provides a completely different, *general* solution to the above problems: Based on redefining the Tcl command corresponding to a widget, the main Wcb procedure `wcb::callback` enables you to associate arbitrary commands with some Tk entry, Ttk entry, BWidget Entry, Tk spinbox, Ttk spinbox, Ttk combobox, listbox, tablelist, Ttk treeview, text, and ctext widget operations. These commands will be invoked automatically in the global scope whenever the respective widget operation is executed. You can request that these commands be called either before or after executing the respective widget operation, i.e., you can define both **before-** and **after-callbacks**. From within a before-callback, you can cancel the respective widget command by invoking the procedure `wcb::cancel`, or modify its arguments by calling `wcb::extend` or `wcb::replace`.

Besides these (and four other) general-purpose commands, the Wcb package exports four utility procedures for Tk entry, Ttk entry, BWidget Entry, Tk spinbox, Ttk spinbox, and Ttk combobox widgets, as well as some before-`insert` callbacks for Tk entry, Ttk entry, BWidget Entry, Tk spinbox, Ttk spinbox, Ttk combobox, text, and ctext widgets, which you can use unchanged or modify to suit your needs. To learn how to do this, have a look at the Examples section below.

The Wcb package is implemented in pure Tcl/Tk code, which makes it completely platform-independent and very easy to install. It requires version 8.4 or higher of both Tcl and Tk.

How to Get It?

Wcb is available for free download from the Web page

https://www.nemethi.de

The distribution file is `wcb4.2.tar.gz` for UNIX and `wcb4_2.zip` for Windows. These files contain the same information, except for the additional carriage return character preceding the linefeed at the end of each line in the text files for Windows.

Wcb is also included in tklib, which has the address

https://core.tcl.tk/tklib

How to Install It?

Install the package as a subdirectory of one of the directories given by the `auto_path` variable. For example, you can install it as a subdirectory of the `lib` directory within your Tcl/Tk installation.

To install Wcb *on UNIX*, `cd` to the desired directory and unpack the distribution file `wcb4.2.tar.gz`:

```
gunzip -c wcb4.2.tar.gz | tar -xf -
```

On most UNIX systems this can be replaced with

```
tar -zxf wcb4.2.tar.gz
```

Both commands will create a directory named `wcb4.2`, with the subdirectories `demos`, `doc`, and `scripts`.

On Windows, use WinZip or some other program capable of unpacking the distribution file `wcb4_2.zip` into the directory `wcb4.2`, with the subdirectories `demos`, `doc`, and `scripts`.

Notice that in tklib the Wcb `demos` directory is replaced with the subdirectory `wcb` of the `examples` directory. Please take this into account when reading the examples below.

How to Use It?

To be able to access the commands and variables of the Wcb package, your scripts must contain one of the lines

```
package require wcb ?version?
package require Wcb ?version?
```

You can use either one of the two statements above because the file `wcb.tcl` contains both lines

```
package provide wcb ...
package provide Wcb ...
```

You are free to remove one of these two lines from `wcb.tcl` if you want to prevent the package from making itself known under two different names. Of course, by doing so you restrict the argument of `package require`to a single name.

Since the Wcb package is implemented in its own namespace called `wcb`, you must either invoke the

```
namespace import wcb::pattern ?wcb::pattern ...?
```

command to import the *procedures* you need, or use qualified names like `wcb::callback`. In the examples below we have chosen the latter approach.

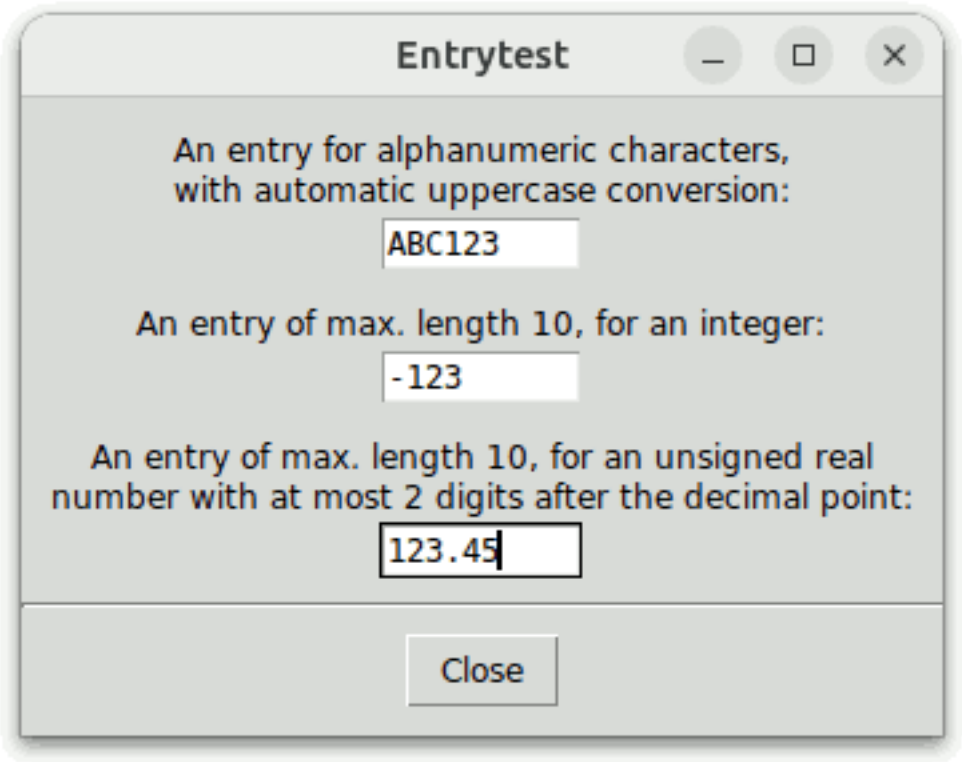
To access Wcb *variables*, you *must* use qualified names. There are only two Wcb variables that are designed to be accessed outside the namespace `wcb`:

- The variable `wcb::version` holds the current version number of the Wcb package.
- The variable `wcb::library` holds the location of the Wcb installation directory.

Examples

Some before-insert Callbacks for entry Widgets

The script `entrytest.tcl` in the `demos` directory creates three entry widgets with the constraints shown in the following figure:



entrytest

For the topmost entry `.e1` we define two before-`insert` callbacks contained in the Wcb package:

```
wcb::callback .e1 before insert wcb::checkStrForAlnum \
                                wcb::convStrToUpper
```

To force the second entry `.e2` to accept only integers of maximal length 10, we use again two before-`insert` callbacks from Wcb:

```
wcb::callback .e2 before insert {wcb::checkEntryLen 10} \
                                 wcb::checkEntryForInt
```

And finally, here are the two callbacks for the third entry widget `.e3`:

```
wcb::callback .e3 before insert {wcb::checkEntryLen 10} \
                                 checkNumber

#
# Callback procedure checkNumber
#
proc checkNumber {w idx str} {
    set newText [wcb::postInsertEntryText $w $idx $str]
    if {![regexp {^[0-9]*\.?[0-9]?[0-9]?$} $newText]} {
        wcb::cancel
    }
}
```

This last example also shows the arguments of the callbacks declared with the `wcb::callback` command: Whenever a callback is invoked, the new name of the original Tcl command for the widget (i.e., the name of the proxy command) as well as the arguments of the respective widget operation are automatically appended to it as parameters. Since we defined `checkNumber` as a before-callback for the `insert` subcommand, its last three arguments must be: the name of the proxy widget command (`w`, which in this example is `::_.e3`), the index (`idx`), and the string (`str`) to be inserted just before the character indicated by the index.

Notice that in the argument list of a Wcb callback, the name of the proxy widget command can be be preceded by any number of additional arguments. The procedure `wcb::checkEntryLen` is an example of such a callback.

The command `wcb::postInsertEntryText` invoked in the procedure `checkNumber` returns the text that would be contained in the entry widget `w` after inserting the string `str` before the character indicated by the index `idx`. If this text is not (the starting part of) an unsigned real number with at most two digits after the decimal point, then we call the procedure `wcb::cancel`, which aborts the `insert` command.

Without the constraint that the content of the third entry must not start with a sign, we could have used the callback procedure `wcb::checkEntryForFixed` instead of `checkNumber`:

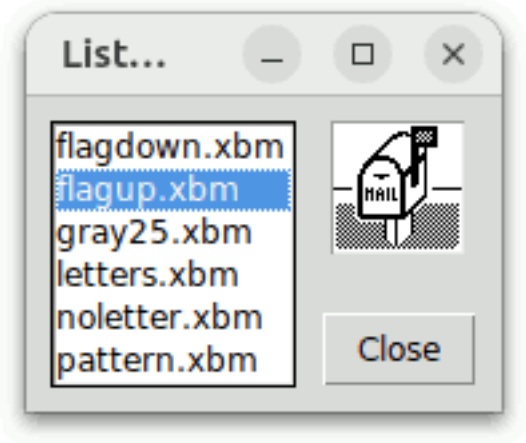
```
wcb::callback .e3 before insert {wcb::checkEntryLen 10} \
                                {wcb::checkEntryForFixed 2}
```

A selset Callback for a listbox Widget

In the case of a listbox, you will probably most often want to define a callback for the `selection set`widget subcommand. In most cases it does not matter whether this is a before- or after-callback. Please note that the `wcb::callback` command expects the abbreviated form `selset` as parameter. Similarly, you must pass `selclear` to this command when defining a callback for the `selection clear`listbox operation.

In the following example we build a listbox `.lb` containing the names of the bitmap files in the subdirectory `images` of the Wcb `demos` directory. (The directory `images` is a copy of the subdirectory of the same name of the Tk 8.6 `demos` directory.) Whenever an item is selected, the callback procedure `showBitmap` will display the corresponding bitmap in the label

`.picture`.



listboxtest1

Here is the relevant code fragment from the script `listboxtest1.tcl`, contained in the `demos` directory:

```
set dirName [file join [file dirname [info script]] images]

#
# Frame .spacer and listbox .lb
#
frame .spacer -width 7p
listbox .lb -height 0 -width 0 -background white
set pattern [file join $dirName *.xbm]
foreach pathName [lsort [glob $pattern]] {
    .lb insert end [file tail $pathName]
}

#
# Label .picture
#
label .picture -relief sunken -background white

#
# Define a before-selset callback for .lb
#
wcb::callback .lb before selset showBitmap

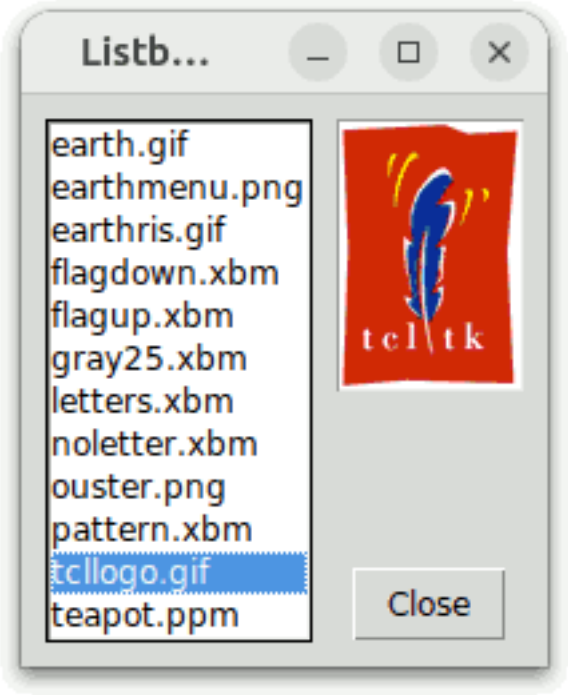
#
# Callback procedure showBitmap
#
proc showBitmap {w first args} {
    global dirName
    set pathName [file join $dirName [$w get $first]]
    .picture configure -bitmap @$pathName
}
```

Recall that the `selection set`listbox operation takes as arguments one or two indices, which will be passed automatically to the callback as parameters, along with the new name of the original Tcl command associated with the listbox widget (i.e., the name of the proxy command).

For this reason, the arguments of the callback procedure `showBitmap` are: the name of the proxy widget command (`w`, which in this example is `::_.lb`), the first index (`first`), as well as the `args` keyword representing the empty list or the optional second index passed to the `selection set`command.

An activate Callback for a listbox Widget

The listbox used in the preceding example has the default selection mode `browse`, hence the before-`selset` callback `showBitmap` will be fired every time the mouse is dragged from one element to another, with button 1 down. But what happens if we want to display not only the bitmaps but also the photo images contained in the subdirectory `images` of the Wcb `demos` directory? Loading a photo image is a much more complex operation than loading a bitmap, which can have the effect that some images cannot be displayed quickly enough to follow the mouse when browsing with it within the listbox.



listboxtest2

To solve this problem, we can either change the selection mode to have the less common value `single`, or arrange for the images not to be displayed when browsing with the mouse but when releasing its button 1. The second method can be implemented with the aid of an `activate` callback, as shown in the following code fragment taken from the script `listboxtest2.tcl`, contained in the `demos` directory:

```
set dirName [file join [file dirname [info script]] images]
image create photo photoImage

#
# Frame .spacer and listbox .lb
#
frame .spacer -width 7p
listbox .lb -height 0 -width 0 -background white
set pattern [file join $dirName *]
```

```
foreach pathName [lsort [glob $pattern]] {
    .lb insert end [file tail $pathName]
}

#
# Label .picture
#
label .picture -relief sunken -background white

#
# Define a before-activate callback for .lb
#
wcb::callback .lb before activate showPicture

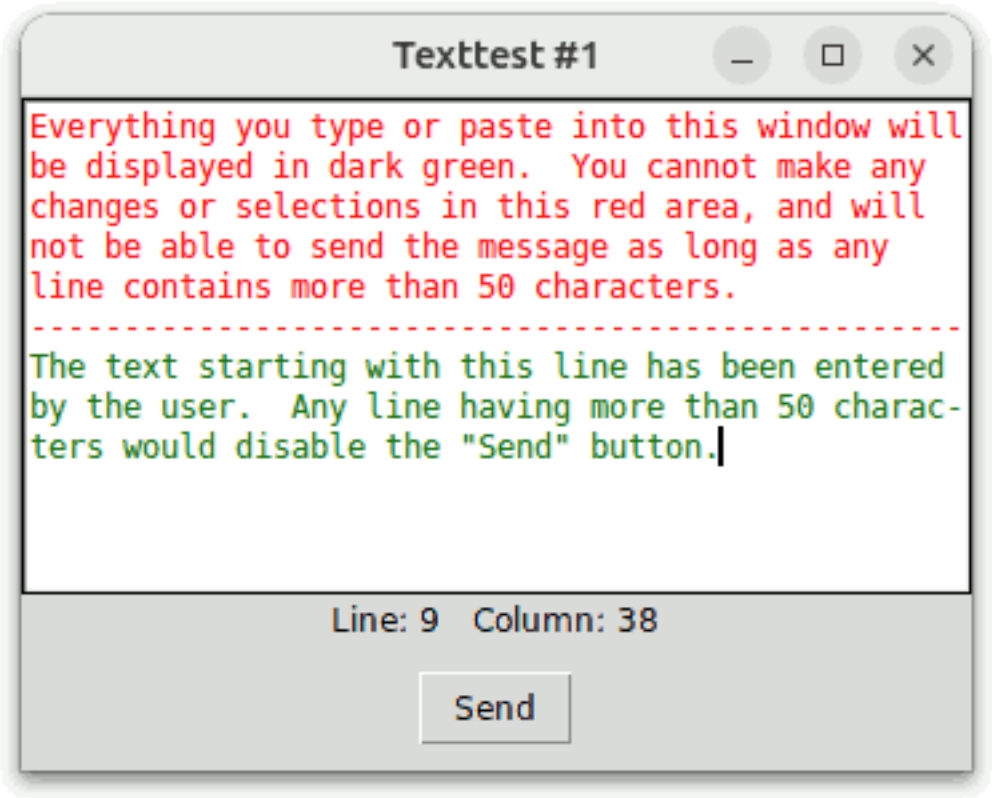
#
# Callback procedure showPicture
#
proc showPicture {w idx} {
    set leafName [$w get $idx]

    #
    # When traversing the listbox with the arrow keys, the value
    # of idx can become -1 or the number of listbox elements,
    # hence the value of leafName can be an empty string:
    #
    if {$leafName eq ""} {
        return ""
    }

    global dirName
    set pathName [file join $dirName $leafName]
    if {[regexp {^\.(bmp|xbm)$} [file extension $pathName]]} {
        .picture configure -bitmap @$pathName -image ""
    } else {
        .picture configure -bitmap "" -image ""
        catch {
            photoImage configure -file $pathName
            .picture configure -bitmap "" -image photoImage
        }
    }
}
```

Seven Callbacks for a text Widget

The script `texttest1.tcl` in the `demos` directory creates the text widget shown in the following figure:



texttest1

Here is the relevant code fragment:

```
#
# Text .txt
#
set width 50
text .txt -width $width -height 12 -setgrid true -wrap none -background white \
          -font TkFixedFont
.txt tag configure prog -foreground red
.txt tag configure user -foreground DarkGreen
.txt insert end "Everything you type or paste into this window will\n"  prog
.txt insert end "be displayed in dark green.  You cannot make any\n"    prog
.txt insert end "changes or selections in this red area, and will\n"    prog
.txt insert end "not be able to send the message as long as any\n"      prog
.txt insert end "line contains more than $width characters.\n"          prog
.txt insert end "--------------------------------------------------\n"  prog
set limit [.txt index insert]

#
# Label .pos displaying the current cursor position
#
label .pos -textvariable pos

#
# Button .send (actually, it does not send anything)
#
button .send -text Send -command exit
```

```
#
# Define 5 before- and 2 after-callbacks for .txt
#
wcb::callback .txt before insert protectRedArea changeColor
wcb::callback .txt before delete protectRedArea
wcb::callback .txt before selset protectRedArea
wcb::callback .txt before motion displayPos
wcb::callback .txt  after insert "checkLines $width"
wcb::callback .txt  after delete "checkLines $width"

#
# Callback procedure protectRedArea
#
# The parameters following w can be interpreted either as
# "index string ?tagList string tagList ...?" (for an insert
# callback), or as "from ?to?" (for a delete callback),
# or as "from ?to from to ...?" (for a selset callback).
#
proc protectRedArea {w idx args} {
    global limit
    if {[$w compare $idx < $limit]} {
        wcb::cancel
    }
}

#
# Callback procedure changeColor
#
proc changeColor {w args} {
    wcb::extend user
}

#
# Callback procedure displayPos
#
proc displayPos {w idx} {
    set index [$w index $idx]
    scan $index "%d.%d" line column
    incr column

    global pos
    set pos [format "Line: %d   Column: %d" $line $column]
}

#
# Callback procedure checkLines
#
# The parameter args can be interpreted both as "index
# string ?tagList string tagList ...?" (for an insert
# callback) and as "from ?to?" (for a delete callback).
#
proc checkLines {maxCharsPerLine w args} {
    #
```

```
    # Display the new cursor position
    #
    displayPos $w insert

    #
    # Disable or enable the .send button
    #
    scan [$w index end] "%d" lastLine
    for {set line 1} {$line < $lastLine} {incr line} {
        scan [$w index $line.end] "%d.%d" dummy charsInLine
        if {$charsInLine > $maxCharsPerLine} {
            .send configure -state disabled
            return ""
        }
    }
    .send configure -state normal
}

. . .

displayPos .txt insert
focus .txt
```

The procedure `protectRedArea` is a before-`insert`, before-`delete`, and before-`selset` callback. It checks whether the attempted change would affect the text area displayed in red; if this is the case, it calls the procedure `wcb::cancel`, which aborts the `insert`, `delete`, or `tag add sel`command, respectively.

The before-`insert` callback `changeColor` invokes the `wcb::extend` command to append the `user` tag to the argument list of the `insert` command, thus changing the foreground color of the characters entered by the user to `DarkGreen`.

The procedure `displayPos` displays the line and column corresponding to the index passed to it as its second argument. This index will be the target position of the insertion cursor when the procedure is triggered automatically as a before-`motion` callback. As seen in the `checkLines` procedure discussed below, it is also invoked after performing an `insert` or `delete` operation; in that case, its second argument will be the new position of the insertion cursor after the execution of `insert` or `delete`. In this way, we are able to keep track completely of the position of the insertion cursor.

It is interesting to see what happens if we register `displayPos` as an after- instead of before-`motion` callback. Well, in that case the procedure would have to ignore its second argument and we would have to replace the line

```
set index [$w index $idx]
```

with

```
set index [$w index insert]
```

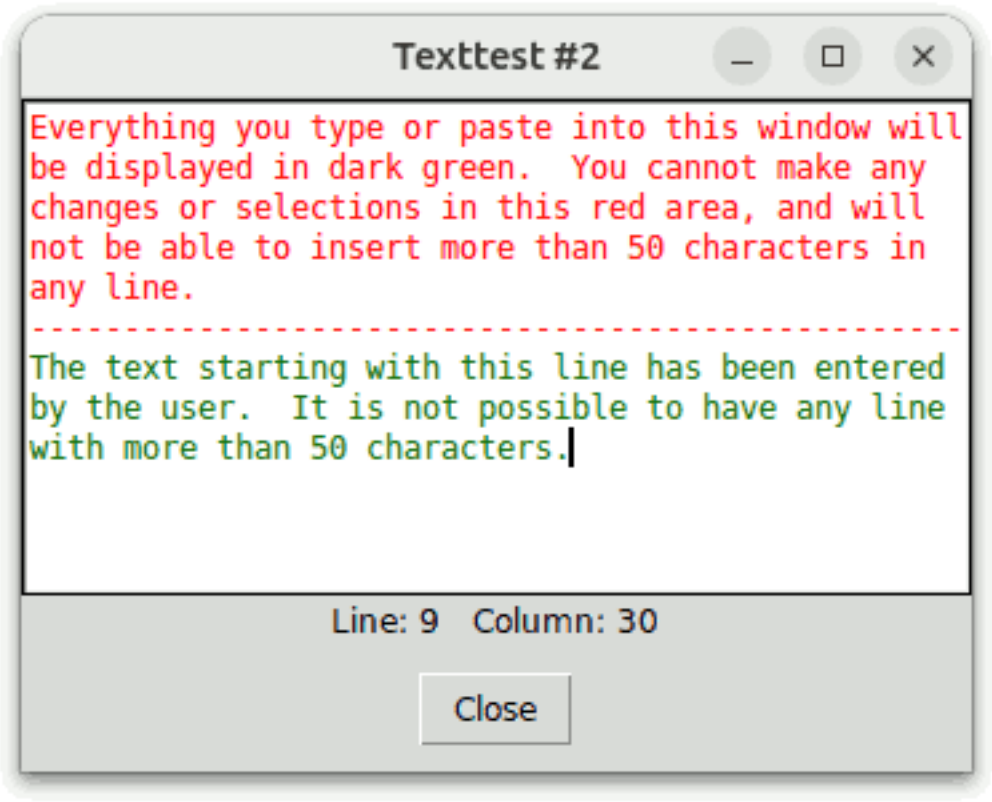
The reason is that the value of the `idx` argument passed to `displayPos` can be, for instance, `insert+1c`, where `insert` means the position of the insertion cursor *before* moving it forward by one character. The after-`motion` callback is, however, triggered *after* the insertion cursor has been moved, and at that time the `insert` mark already points to the new cursor position. For

this reason, `[$w index $idx]`is not adequate to retrieve the position of the insertion cursor within an after-`motion` callback.

Our last procedure `checkLines` is both an after-`insert` and after-`delete` callback. After calling `displayPos` to display the new cursor position, it disables or enables the `.send` button, depending upon whether any line of the text widget contains more than `$maxCharsPerLine` characters.

Notice that we could also have defined before-`replace` and after-`replace` callbacks for the `replace` text widget subcommand, introduced in Tk version 8.6. There is, however, no need for it, because the default text widget bindings currently don't make use of this subcommand.

Instead of just disabling the `.send` button if any line has more than `$maxCharsPerLine` characters, we can even prevent the user from entering lines that are longer than allowed. The script `texttest2.tcl` in the `demos` directory shows how this can be achieved by combining widget callbacks with the undo mechanism for text widgets. This script creates the text widget shown in the following figure:



texttest2

Most of the code contained in the script `texttest2.tcl` is identical to the one in the previous example. The main difference is a new line activating the undo mechanism for the text widget `.txt` and a new version of the callback procedure `checkLines`:

```
. . .

.txt configure -undo yes

. . .
```

```
proc checkLines {maxCharsPerLine w args} {
    #
    # Undo the last insert or delete action if necessary
    #
    scan [$w index end] "%d" lastLine
    for {set line 1} {$line < $lastLine} {incr line} {
        scan [$w index $line.end] "%d.%d" dummy charsInLine
        if {$charsInLine > $maxCharsPerLine} {
            $w edit undo
            bell
            break
        }
    }

    #
    # Clear the undo and redo stacks, and display the new cursor position
    #
    $w edit reset
    displayPos $w insert
}

. . .
```

This version of the callback `checkLines` undoes the last edit action if any text line contains more characters than the allowed maximum. For this reason, we had to move the invocation of the `displayPos` procedure to the end of the callback, because the `edit undo`command might change the position of the insertion cursor.

Note that we could have implemented this example also without making use of the undo mechanism for text widgets, by saving the last accepted contents of the widget, along with the cursor position, and restoring them in case any line gets longer than allowed.

wcbRef

Keywords: wcbRef

Wcb Command Reference

For Wcb Version 4.2

by

Csaba Nemethi
csaba.nemethi@t-online.de

Contents

- The wcb::callback Command
- The wcb::cbappend Command
- The wcb::cbprepend Command
- The wcb::cancel Command
- The wcb::canceled Command

- The wcb::extend Command
- The wcb::replace Command
- The wcb::pathname Command
- The wcb::changeEntryText Command
- The wcb::postInsertEntryLen Command
- The wcb::postInsertEntryText Command
- The wcb::postDeleteEntryText Command
- Before-insert Callbacks for entry, spinbox, and combobox Widgets
- Before-insert Callbacks for text and ctext Widgets

The wcb::callback Command

NAME

`wcb::callback` – Retrieve, set, and remove widget callbacks

SYNOPSIS

```
wcb::callback widgetName before|after option ?callback callback ...?
```

DESCRIPTION

Retrieves, sets, or removes the callbacks for the Tk or Ttk entry, BWidget Entry, Tk or Ttk spinbox, Ttk combobox, listbox, tablelist, Ttk treeview, text, or ctext widget `widgetName`, the argument `before` or `after`, and the command corresponding to `option`. The values of the `option` argument can be:

- `insert`, `delete`, or `motion`, for a Tk or Ttk entry, BWidget Entry, Tk or Ttk spinbox, Ttk combobox, text, or ctext widget;
- `replace`, for a text or ctext widget;
- `activate`, for a listbox, tablelist, or Ttk treeview widget;
- `selset` or `selclear`, for a listbox, tablelist, Ttk treeview, text, or ctext widget;
- `seladd` or `seltoggle`, for a Ttk treeview widget;
- `cellselset` or `cellselclear`, for a tablelist or Ttk treeview widget;
- `cellseladd` or `cellseltoggle`, for a Ttk treeview widget;
- `activatecell`, for a tablelist widget.

If no arguments after the `option` parameter are specified, then the procedure just returns the current before- or after-callback list, respectively, for the given widget operation.

Otherwise:

- If at least one of the arguments following the `option` parameter is a nonempty string, then:
- If all arguments following the `option` parameter are empty, then the procedure unregisters all the corresponding before- or after-callbacks for the given widget and returns an empty string.
  - if called for the first time for this widget with at least one nonempty argument following the `option` parameter, then the procedure renames the Tcl command `::widgetName` to `::_widgetName` and builds a new procedure `::widgetName`, in which the execution of the widget operations associated with the above values of `option` is preceded by invocations of the corresponding before-callbacks and followed by calls to the corresponding after-callbacks, in the global scope;
  - it sets the callback list to the one built from these arguments and returns the new list.

When a callback is invoked for the widget `widgetName`, the name of the **proxy command** `::_widgetName` and the command arguments are automatically appended to it as parameters. Use the `wcb::pathname` command to retrieve the original widget path name.

The following table shows the widget subcommands corresponding to the above values of `option`, together with the arguments of these subcommands:

Widget	**option**	**Subcommand**	**Arguments**
Tk entry, ttk::entry, BWidget Entry, Tk spinbox, ttk::spinbox, or ttk::combobox	insert	insert	index string
delete	delete	from ?to?	
motion	icursor	index	
listbox	activate	activate	index
selset	selection set	first ?last?	
selclear	selection clear	first ?last?	
tablelist::tablelist	activate	activate	index
selset	selection set	first ?last? (indexList or firstIndex lastIndex)	
selclear	selection clear	first ?last? (indexList or firstIndex lastIndex)	
activatecell	activatecell	cellIndex	
cellselset	cellselection set	first ?last? (cellIndexList or firstCell lastCell)	
cellselclear	cellselection clear	first ?last? (cellIndexList or firstCell lastCell)	
ttk::treeview	activate	focus	item
selset	selection set	itemList	
seladd	selection add	itemList	
selclear	selection remove	itemList	
seltoggle	selection toggle	itemList	
cellselset	cellselection set	first ?last? (cellList or firstCell lastCell)	

Widget	option	Subcommand	Arguments
cellseladd	cellselection add	first ?last? (cellList or firstCell lastCell)	
cellselclear	cellselection remove	first ?last? (cellList or firstCell lastCell)	
cellseltoggle	cellselection toggle	first ?last? (cellList or firstCell lastCell)	
text or ctext	insert	insert	index string ?tagList string tagList ...?
delete	delete	from ?to?	
replace	replace	from to string ?tagList string tagList ...?	
motion	mark set insert	index	
selset	tag add sel	from ?to from to ...?	
selclear	tag remove sel	from ?to from to ...?	

REMARKS:

1. You may abbreviate the words `before`, `after`, `insert`, `delete`, `replace`, `motion`, and `activate` (the latter not for tablelist widgets) to a minimum of one character. Similarly, the first four characters of the words `selset`, `selclear`, `seladd`, and `seltoggle`, the first eight characters of `cellselset`, `cellselclear`, `cellseladd`, and `cellseltoggle`, and the first nine characters of `activatecell` are sufficient for Wcb to recognize these options.
2. After a successful invocation of this command with at least one nonempty callback following the `option` argument, you can use either the new procedure `widgetName` or the proxy command `::_widgetName` to perform any valid operation on the widget `widgetName`. Use the proxy command `::_widgetName` if you want to prevent the callbacks from being invoked when executing the respective widget subcommand.
3. When destroying a widget `widgetName` for which `wcb::callback` has replaced the corresponding Tcl command with a new procedure, the proxy command `::_widgetName` is deleted automatically by the Tcl interpreter (this is not true in the case of a BWidget Entry, tablelist, or ctext widget). The new widget procedure `widgetName` would persist, but Wcb arranges for it to be deleted from within a cleanup script bound to the `<Destroy>` event. This cleanup script is associated with a binding tag called `WcbCleanup`, which is appended to the list of binding tags of the widget the first time when registering some callbacks for it. (In the case of a BWidget Entry, tablelist, or ctext widget, this script also deletes the proxy command `::_widgetName`.)
4. The cleanup script mentioned above also unregisters all callbacks defined for `widgetName`, thus ensuring that a widget with the same path created later will not inherit them from the widget just deleted (this can be important in some applications). For this reason, you should be careful not to remove `WcbCleanup` from the list of binding tags of the given widget!

KEYWORDS

callback, widget, Tk entry, Ttk entry, BWidget Entry, Tk spinbox, Ttk spinbox, Ttk combobox, listbox, tablelist, Ttk treeview, text, ctext

---

The wcb::cbappend Command

NAME

`wcb::cbappend` – Append to a callback list

SYNOPSIS

```
wcb::cbappend widgetName before|after option ?callback callback ...?
```

DESCRIPTION

This command is almost identical to `wcb::callback`. The only difference is that `wcb::cbappend` *appends* the arguments specified after the `option` parameter to the current callback list (if present), while `wcb::callback` *replaces* the old callbacks with these arguments.

KEYWORDS

callback, append, widget

The wcb::cbprepend Command

NAME

`wcb::cbprepend` – Prepend to a callback list

SYNOPSIS

```
wcb::cbprepend widgetName before|after option ?callback callback ...?
```

DESCRIPTION

This command is almost identical to `wcb::callback`. The only difference is that `wcb::cbprepend` *prepends* the arguments specified after the `option` parameter to the current callback list (if present), while `wcb::callback` *replaces* the old callbacks with these arguments.

KEYWORDS

callback, prepend, widget

The wcb::cancel Command

NAME

`wcb::cancel` – Cancel a widget command

SYNOPSIS

```
wcb::cancel ?script?
```

DESCRIPTION

This procedure is designed to be invoked from a before-callback for a widget command. It cancels the execution of that command and of the remaining callbacks, and evaluates the `script` argument in the global scope. If this argument is not specified, it defaults to the

`bell` command.

The return value is the one obtained from `script` if this argument is specified and nonempty. Otherwise, the command returns an empty string.

KEYWORDS

cancel, command, widget

The wcb::canceled Command

NAME

`wcb::canceled` – Query the canceled status of a widget command

SYNOPSIS

```
wcb::canceled widgetName option
```

DESCRIPTION

Returns the value `1` if the most recent invocation of the widget operation corresponding to `widgetName` and `option` has been aborted by some before-callback by invoking `wcb::cancel`; otherwise, the return value is `0`. The arguments must fulfil the same restrictions as in the case of the `wcb::callback` command.

KEYWORDS

cancel, command, widget

The wcb::extend Command

NAME

`wcb::extend` – Extend the argument list of a widget command

SYNOPSIS

```
wcb::extend ?arg arg ...?
```

DESCRIPTION

This procedure is designed to be invoked from a before-callback for a widget command. It appends the values given in the optional `arg` parameters to the argument list of that command. The new argument list will be passed to the remaining callbacks for that command, too.

This procedure simply passes its parameters to the `lappend` command, called for the argument list of the respective widget operation.

KEYWORDS

extend, argument, command, widget

The wcb::replace Command

NAME

`wcb::replace` – Replace arguments of a widget command with new ones

SYNOPSIS

```
wcb::replace first last ?arg arg ...?
```

DESCRIPTION

This procedure is designed to be invoked from a before-callback for a widget command. It replaces the arguments having the indices `first` through `last` of that command with the optional `arg` parameters. The new argument list will be passed to the remaining callbacks for that command, too. The arguments are numbered from `0` (see the table in the description of the `wcb::callback` command).

This procedure simply passes its parameters to the `lreplace` command, called for the argument list of the respective widget operation.

KEYWORDS

replace, argument, command, widget

The wcb::pathname Command

NAME

`wcb::pathname` – Query the path name of the widget corresponding to a Tcl command name

SYNOPSIS

```
wcb::pathname proxyCmd
```

DESCRIPTION

This procedure returns the path name of the widget corresponding to the Tcl command name `proxyCmd` passed to a callback.

When a before- or after-callback for a widget is invoked, the name of the proxy command associated with that widget is automatically appended to it as parameter. This procedure can be used within a callback to retrieve the path name of the widget from the command name passed to the callback as argument. It simply returns the string range obtained from `proxyCmd` by removing the `"::_"` prefix.

KEYWORDS

path name, command, widget

The wcb::changeEntryText Command

NAME

`wcb::changeEntryText` – Change the text of a Tk or Ttk entry, BWidget Entry, Tk or Ttk spinbox, or Ttk combobox widget

SYNOPSIS

```
wcb::changeEntryText widgetName string
```

DESCRIPTION

Replaces the text of the Tk or Ttk entry, BWidget Entry, Tk or Ttk spinbox, or Ttk combobox widget `widgetName` with `string`, by using the `delete` and `insert` operations. If the first subcommand is canceled by some before-`delete` callback then the procedure returns without inserting the new text; if the second operation is canceled by some before-`insert`

callback then the command restores the original contents of the widget.

The procedure keeps the position of the insertion cursor. The return value is `1` on success and `0` on failure, i.e., if one of the attempted operations was canceled by some before-callback.

KEYWORDS

Tk entry, Ttk entry, BWidget Entry, Tk spinbox, Ttk spinbox, Ttk combobox, widget

---

The wcb::postInsertEntryLen Command

NAME

`wcb::postInsertEntryLen` – Query the would-be length of the text in a Tk or Ttk entry, BWidget Entry, Tk or Ttk spinbox, or Ttk combobox widget after text insertion

SYNOPSIS

```
wcb::postInsertEntryLen widgetName string
```

DESCRIPTION

Returns the length of the text that would be contained in the Tk or Ttk entry, BWidget Entry, Tk or Ttk spinbox, or Ttk combobox widget `widgetName` after inserting `string`.

KEYWORDS

insert, Tk entry, Ttk entry, BWidget Entry, Tk spinbox, Ttk spinbox, Ttk combobox, widget

---

The wcb::postInsertEntryText Command

NAME

`wcb::postInsertEntryText` – Query the would-be text of a Tk or Ttk entry, BWidget Entry, Tk or Ttk spinbox, or Ttk combobox widget after text insertion

SYNOPSIS

```
wcb::postInsertEntryText widgetName index string
```

DESCRIPTION

Returns the text that would be contained in the Tk or Ttk entry, BWidget Entry, Tk or Ttk spinbox, or Ttk combobox widget `widgetName` after inserting `string` before the character indicated by `index`.

KEYWORDS

insert, Tk entry, Ttk entry, BWidget Entry, Tk spinbox, Ttk spinbox, Ttk combobox, widget

---

The wcb::postDeleteEntryText Command

NAME

`wcb::postDeleteEntryText` – Query the would-be text of a Tk or Ttk entry, BWidget Entry, Tk or Ttk spinbox, or Ttk combobox widget after text deletion

SYNOPSIS

```
wcb::postDeleteEntryText widgetName from ?to?
```

DESCRIPTION

Returns the text that would be contained in the Tk or Ttk entry, BWidget Entry, Tk or Ttk spinbox, or Ttk combobox widget `widgetName` after deleting the characters starting with the one indicated by the index `from` and stopping just before `to`. If `to` is not specified then the return value is the text that would be contained in the widget after deleting the single character given by `from`.

REMARK: This command has a variable number (2 or 3) of arguments, because the `delete` subcommand of the Tcl command associated with `widgetName` expects either one or two indices as arguments. For this reason, the correct way to invoke this command from within a before-`delete` callback is as shown in the following example:

```
proc myBeforeDeleteCallback {w args} {
    #
    # Get the text that would be contained in the widget after
    # deleting the characters specified by $args, which stands
    # for the one or two arguments passed to delete; pass these
    # arguments to wcb::postDeleteEntryText by expanding $args
    #
    set newText [eval [list wcb::postDeleteEntryText $w] $args]
    if {!some_condition_on_$newText} {
        wcb::cancel
    }
}
```

The following alternative, more elegant solution requires Tcl/Tk 8.5 or later:

```
proc myBeforeDeleteCallback {w args} {
    # . . .
    set newText [wcb::postDeleteEntryText $w {*}$args]
    . . .
}
```

KEYWORDS

delete, Tk entry, Ttk entry, BWidget Entry, Tk spinbox, Ttk spinbox, Ttk combobox, widget

---

Before-`insert` Callbacks for entry, spinbox, and combobox Widgets

NAME

`wcb::checkStrFor*`, `wcb::convStrTo*`, `wcb::checkEntryFor*`, `wcb::checkEntryLen` – Before-`insert` callbacks for Tk entry, Ttk entry, BWidget Entry, Tk spinbox, Ttk spinbox, and Ttk combobox widgets

SYNOPSIS

```
wcb::checkStrForRegExp  exp w idx str

wcb::checkStrForAlpha       w idx str
wcb::checkStrForNum         w idx str
wcb::checkStrForAlnum       w idx str

wcb::convStrToUpper         w idx str
```

```
wcb::convStrToLower         w idx str

wcb::checkEntryForInt       w idx str
wcb::checkEntryForUInt  max w idx str
wcb::checkEntryForReal      w idx str
wcb::checkEntryForFixed cnt w idx str

wcb::checkEntryLen      len w idx str
```

DESCRIPTION

The `wcb::checkStrForRegExp` callback checks whether the string `str` to be inserted into the Tk or Ttk entry, BWidget Entry, Tk or Ttk spinbox, or Ttk combobox widget `w` is matched by the regular expression `exp`; if not, it cancels the `insert` operation.

The three other `wcb::checkStrFor*` callbacks check whether the string `str` to be inserted into the Tk or Ttk entry, BWidget Entry, Tk or Ttk spinbox, or Ttk combobox widget `w` is alphabetic, numeric, or alphanumeric, respectively; if not, they cancel the `insert` operation. These procedures just invoke the callback `wcb::checkStrForRegExp`, passing to it the Unicode-based patterns `{^[[:alpha:]]*$}`, `{^[[:digit:]]*$}`, and `{^[[:alnum:]]*$}`.

The `wcb::convStrTo*` callbacks replace the string `str` to be inserted into the Tk or Ttk entry, BWidget Entry, Tk or Ttk spinbox, or Ttk combobox widget `w` with its uppercase or lowercase equivalent, respectively.

The `wcb::checkEntryFor*` callbacks check whether the text contained in the Tk or Ttk entry, BWidget Entry, Tk or Ttk spinbox, or Ttk combobox widget `w` after inserting the string `str` before the character indicated by the index `idx` would represent (the starting part of) an integer number, unsigned integer no greater than `max`, real number, or real number in fixed-point format with at most `cnt` digits after the decimal point, respectively; if not, they cancel the `insert` operation. `max` and `cnt` should be nonnegative numbers or `*`; `max = *`means: no upper bound for the Tk or Ttk entry, BWidget Entry, Tk or Ttk spinbox, or Ttk combobox value, while `cnt = *`stands for an unlimited number of digits after the decimal point.

The `wcb::checkEntryLen` callback checks whether the length of the text contained in the Tk or Ttk entry, BWidget Entry, Tk or Ttk spinbox, or Ttk combobox widget `w` after inserting the string `str` would be greater than `len`; if yes, it cancels the `insert` operation.

These callback procedures are implemented in the file `wcbEntry.tcl`, contained in the `scripts` directory. They return an empty string.

KEYWORDS

callback, insert, Tk entry, Ttk entry, BWidget Entry, Tk spinbox, Ttk spinbox, Ttk combobox, widget

---

Before-insert Callbacks for text and ctext Widgets

NAME

wcb::checkStrsFor*, wcb::convStrsTo* – Before-insert callbacks for text and ctext widgets

SYNOPSIS

```
wcb::checkStrsForRegExp exp w idx args

wcb::checkStrsForAlpha      w idx args
wcb::checkStrsForNum        w idx args
wcb::checkStrsForAlnum      w idx args

wcb::convStrsToUpper        w idx args
wcb::convStrsToLower        w idx args
```

DESCRIPTION

The wcb::checkStrsForRegExp callback checks whether the strings to be inserted into the text or ctext widget w, contained in the list args of the form string ?tagList string tagList ...?, are matched by the regular expression exp; if not, it cancels the insert operation.

The three other wcb::checkStrsFor* callbacks check whether the strings to be inserted into the text or ctext widget w, contained in the list args of the form string ?tagList string tagList ...?, are alphabetic, numeric, or alphanumeric, respectively; if not, they cancel the insert operation. These procedures just invoke the callback wcb::checkStrsForRegExp, passing to it the Unicode-based patterns {^[[:alpha:]\n]*$}, {^[[:digit:]\n]*$}, and {^[[:alnum:]\n]*$}.

The wcb::convStrsTo* callbacks replace the strings to be inserted into the text or ctext widget w, contained in the list args of the form string ?tagList string tagList ...?, with their uppercase or lowercase equivalents, respectively.

These callback procedures are implemented in the file wcbText.tcl, contained in the scripts directory. They return an empty string.

KEYWORDS

callback, insert, text, ctext, widget

License

These pages were converted from the original manual pages to Markdown (TIP 700 format). The text is unchanged; only markup without a place in the format (links without a target, layout images) may be lost. For parts under the Mozilla Public License 1.1 (tDOM) this is the description of modifications required by section 3.3.

This collection is not an official publication of the projects. Each part is under the terms of its original, which follow. The copyright notices of the original pages stay with each page.

Widget callback package Wcb, version 4.2 Copyright (c) 1999-2025 Csaba Nemethi (E-mail: csaba.nemethi@t-online.de)

This library is free software; you can use, modify, and redistribute it for any purpose, provided

that existing copyright notices are retained in all copies and that this notice is included verbatim in any distributions.

This software is distributed WITHOUT ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE.

Index

A

A selset Callback for a listbox Widget 7
An activate Callback for a listbox Widget 9

B

Before-insert Callbacks for text and ctext Widgets 24

I

index 3

S

Seven Callbacks for a text Widget 10
Some before-insert Callbacks for entry Widgets 6
Some Common Problems 3

T

The Wcb Package 4

W

wcb 3
wcb::callback 16
wcb::cancel 19
wcb::canceled 20
wcb::cbappend 19
wcb::cbprepend 19
wcb::changeEntryText 21
wcb::extend 20
wcb::pathname 21
wcb::postDeleteEntryText 22
wcb::postInsertEntryLen 22
wcb::postInsertEntryText 22
wcb::replace 20
wcbRef 15